

Giolit Program Verifier 7.0 — README

Installation, first steps, commands and the contents of this package

Contents

What this is	1
This package	2
1. Requirements	2
2. Installation	3
3. First steps	3
4. Commands	3
5. Writing programs for verification	4
6. The benchmarks and results in this package	5
7. Integrating the verifier	5
8. Licensing	5
9. Troubleshooting	6
10. Support	6

What this is

Mathematical proof that your C code is free of run-time errors — or the exact input that makes it fail.

The Giolit Program Verifier analyses C programs and decides, for every operation that can fail at run time, whether it can fail for **any** input:

Result	Meaning
PROVEN	No execution, for any input, violates the check. The proof is included.
VIOLATED	A real execution violates it: the failing input and every step are included.

Result	Meaning
UNREACHABLE	The operation never runs (dead code, or not called from the entry function).
UNKNOWN	Not decided within the time budget (see the tutorial, “When the result is UNKNOWN”).

It checks assertions, signed integer overflow, division by zero, invalid shifts, array bounds, pointer dereferences, *free* () and memory leaks, in sequential, recursive and multithreaded (POSIX threads) programs. Each run can produce a PDF certificate or report with the proof drawn on the program’s control-flow graph, plus JSON, SARIF and the complete proof.

This is the **trial edition**: 1000 runs within 50 days on one computer. Documents are marked “Trial version”. A purchased license turns them into professional documents with your license number (section 8).

This package

Folder	Contents
code/	The verifier: <code>bin/</code> (the <code>v7</code> command), <code>lib/</code> (the verifier libraries), <code>include/</code> (the public C++ API), <code>share/giolit/</code> (the logo and the twelve tutorial examples), <code>sdk-example/</code> (five example clients of the API and the C programs they verify), and <code>install.sh / uninstall.sh</code>
documents/	This README, the Tutorial (start there), the Product Overview , and the License Terms
benchmarks/	Verification benchmarks with a known answer, by category : <code>sequential</code> , <code>recursive</code> , <code>concurrent</code> , <code>memory</code> , <code>industrial</code> , <code>projects</code> , <code>tutorial-examples</code> . <code>MANIFEST.tsv</code> lists every verification: the expected result, the options and the files
results/	The verifier’s result for every benchmark, in the same hierarchy: one folder per verification with the PDF, the console output, <code>results.json</code> , <code>results.sarif</code> , <code>proof.json</code> and <code>reproduce.sh</code> . <code>SUMMARY.csv</code> lists them all
SHA256SUMS	Checksums of every file in the package

The libraries hold the verification engine; only their public entry points are visible, and each checks a license token. The public headers in `code/include` are the documented interface.

1. Requirements

Requirement	Details
Operating system	Linux x86-64: Ubuntu 24.04 LTS or newer (glibc 2.38, GCC 13 runtime, LLVM 18)
Memory and disk	4 GB RAM minimum, 8 GB recommended; 60 MB for the verifier, 1.5 GB for the optional LaTeX packages
LLVM / Clang 18	<code>libllvm18</code> , <code>libclang-cpp18</code> , <code>libclang-common-18-dev</code>

Requirement	Details
C library headers	libc6-dev (the programs you verify are compiled against them)
OpenSSL 3	libssl3 (license checks)
PDF documents (<i>optional</i>)	LuaLaTeX: texlive-luatex texlive-latex-recommended texlive-latex-extra texlive-pictures texlive-fonts-extra
PDF protection (<i>licensed documents</i>)	qpdf (or mupdf-tools)

On Ubuntu 24.04:

```
sudo apt update
sudo apt install libllvm18 libclang-cpp18 libclang-common-18-dev libc6-dev libssl3
sudo apt install texlive-luatex texlive-latex-recommended texlive-latex-extra \
    texlive-pictures texlive-fonts-extra qpdf          # PDFs (recommended)
```

Without LuaLaTeX the verifier works fully (console, JSON, SARIF); only the PDF documents are not produced.

2. Installation

```
tar xzf giolit-verifier-7.0.0-trial.tar.gz
cd giolit-verifier-7.0.0-trial/code
./install.sh          # for you: ~/.local/giolit-v7, command in ~/.local/bin
sudo ./install.sh --system # or for everyone: /opt/giolit-v7, command in /usr/local/bin
```

install.sh copies the verifier, puts v7 on your PATH, checks the system libraries (and prints the apt command for any that are missing), and proves the first example as a self-test (one trial run). --prefix DIR installs into any folder. To remove it: ~/.local/giolit-v7/uninstall.sh; your license state is kept.

Check the installation:

```
v7 --version      # v7 7.0.0 (trial version)
v7 --license      # runs and days left (uses no run)
```

3. First steps

```
cp -r ~/.local/giolit-v7/share/giolit/examples ~/giolit-examples
cd ~/giolit-examples
v7 01_loop_invariant.c          # ALL CHECKS PROVEN
v7 02_loop_bug.c               # VIOLATIONS FOUND, input n = 8
v7 --report=out/01 01_loop_invariant.c # out/01/certificate.pdf and its evidence
```

The **Tutorial** in documents/ walks through these and all other examples, explains every result and document, and shows how to verify your own code.

4. Commands

```
v7 [options] file.c [file.c ...]
```

Several files are compiled and linked together, as in a normal build.

Option	Meaning	Default
<code>--checks=LIST</code>	Check kinds, comma-separated, or a group (below)	all
<code>--entry=NAME</code>	The function to verify from; its parameters are arbitrary values	main
<code>--timeout=SECONDS</code>	Total time budget	60
<code>--report=DIR</code>	Write the bundle: PDF, JSON, SARIF, proof, sources	—
<code>--issued-to=NAME</code>	The recipient named on the PDF (only with <code>--report</code>)	—
<code>--keep-tex</code>	Keep the document's LaTeX source next to the PDF (only with <code>--report</code>)	off
<code>--json=FILE, --sarif=FILE, --proof=FILE</code>	Per-check results, SARIF 2.1.0, the full proof	—
<code>--verbose</code>	Complete counterexample traces	off
<code>-I DIR, -D NAME[=V], -std=...</code>	Passed to the C compiler, as in your build	—
<code>--license</code>	The license: runs and days left (uses no run); with <code>--verbose</code> also the folder that holds it	—
<code>--activate=FILE</code> or <code>--activate FILE</code>	Apply a purchased license (section 8)	—
<code>--name=, --address=, --password=</code>	The buyer's details for <code>--activate</code> , instead of answering the prompts	asked
<code>--benchmark file.c [maxIterations] [seconds]</code>	One SV-COMP style verdict: SAFE, UNSAFE or UNKNOWN; the optional numbers cap the iterations and the seconds; check kinds from <code>V7_CHECKS</code> (default <code>assertion</code>), entry from <code>V7_ENTRY</code>	off
<code>--version, --help</code>	The version (and whether a purchased license is in force); the options	—

Check kinds: `assertion` (CWE-617), `overflow` (CWE-190), `div-by-zero` (CWE-369), `shift` (CWE-1335), `array-bounds` (CWE-787), `valid-deref` (CWE-476/416), `valid-free` (CWE-415/761), `valid-memtrack` and `valid-memcleanup` (CWE-401). Groups: `all`; `arithmetic` = `overflow`, `div-by-zero`, `shift`; `memsafety` = `valid-deref`, `valid-free`, `valid-memtrack`; `rte` = `all` but `assertion`.

Exit codes: 0 all checks proven, 1 a check violated, 2 checks unknown, 3 usage or input error, 4 no valid license.

Environment variables (advanced): `GIOLIT_HOME` (license state, default `~/.giolit/v7`), `GIOLIT_REPORT_HOSTNAME` (host name shown in reports), `V7_CLANG_ARGS` (extra compiler arguments), `V7_CHECKS` and `V7_ENTRY` (check kinds and entry function in `--benchmark` mode).

5. Writing programs for verification

- **Inputs:** results of functions without a body (for example `__VERIFIER_nondet_int()`), the entry function's parameters, `volatile` variables and external data can have any value.
- **Assumptions:** `__VERIFIER_assume(cond)` restricts the inputs.
- **Properties:** state what must hold with `assert(cond)` or `__VERIFIER_assert(cond)`; the run-time checks are automatic.

```
extern int __VERIFIER_nondet_int(void);
```

```
extern void __VERIFIER_assume(int);
extern void __VERIFIER_assert(int);
```

6. The benchmarks and results in this package

Every benchmark in `benchmarks/` has a known answer, and `results/` holds the verifier's result for it — each one a correct SAFE (all checks proven) or UNSAFE (a real violation found), produced by this version with a 60-second budget. To reproduce one, run its command (the `command` column of `results/SUMMARY.csv`) in its benchmark folder (the `directory` column):

```
cd benchmarks/sequential
v7 --checks=assertion loop-acceleration__const_1-1.c
```

or run `sh reproduce.sh` inside its results folder, which holds a copy of the program. The categories:

Category	What it is
sequential, recursive, concurrent	Programs from the SV-COMP competition benchmarks (reach-safety), and Giolit's thread suite in <code>concurrent/giolit-thread-suite</code>
memory	Heap and pointer programs: s = safe, u = with a memory error
industrial	Embedded C modules (CAN decoding, CRC, cruise control, infusion pump, motor control, sensor voting, wheel speed), as written and with a seeded bug (<code>-DBUG</code>)
projects	Multi-file programs, correct and with a bug
tutorial-examples	The tutorial's examples

7. Integrating the verifier

- **CI:** gate the build on the exit code, and upload `--sarif` output to GitHub code scanning or GitLab (tutorial, section “Continuous integration”).
- **SDK:** `code/sdk-example/` has five clients of the public C++ API: `verify_file` (verify a C program), `unit_check` (verify functions one by one), `certify` (write the certificate bundle), `license_info` and `ci_gate`. `./build.sh` builds them; the tutorial, section “Embedding the verifier: the SDK”, walks through the first two.

8. Licensing

The trial allows 1000 runs within 50 days of the first run; this build also stops accepting trial runs 100 days after it was made. `v7 --license` shows where you stand. The License Terms are in `documents/`.

Purchasing a license. Write to **business.giolitlabs@gmail.com** or visit **www.giolit.com**. Licenses are offered by period and by volume of runs, including unlimited runs for a period; verification services, certificates for safety standards and training are available alongside.

Activating it. You receive a license file, a license number and a password. Activate once:

```
v7 --activate acme.lic      # asks for the buyer's name, address and password
v7 --license               # professional license GIOLIT-PRO-..., licensed to ...
```

Or without prompts (scripts, CI):

```
v7 --activate=acme.lic --name="Acme Ltd" --address="1 Main St, Springfield" --password=...
```

From then on, documents carry your license number and company name at the top instead of the trial marks, and each PDF is protected by its own password, written to `certificate-password.txt` in the same folder.

9. Troubleshooting

Symptom	Remedy
error while loading shared libraries: libLLVM.so.18.1	Install <code>libllvm18 libclang-cpp18</code> (section 1)
cannot analyse the input: clang: compilation of ... failed	The program does not compile: pass your build's <code>-I/-D</code> flags; install <code>libc6-dev</code> and <code>libclang-common-18-dev</code>
the PDF was not produced: lualatex is not installed	Install the LaTeX packages (section 1); the rest of the bundle (JSON, SARIF, proof, sources) is complete
the PDF was not produced: lualatex failed	Install the LaTeX packages (section 1); the <code>.log</code> next to the bundle names the missing one
the PDF was not produced: cannot password-protect the PDF	Install <code>qpdf</code> ; licensed certificates are only written protected
the name, address or password does not match this license	Enter them as on your purchase letter: letter case, spacing and commas do not matter; the password must match exactly
Exit code 4	The trial's runs or days are used, or this build is older than 100 days: <code>v7 --license</code> says which
Many checks UNKNOWN	Tutorial, "When the result is UNKNOWN"

10. Support

Giolit Labs, Ramavarmapuram P.O., Thrissur, Kerala, India — www.giolit.com — business.giolitlabs@gmail.com. Please include the output of `v7 --version` and `v7 --license`, and the result bundle if you can share it.